

CM01 - Introduction, Linux, Shell

Course organization, OS model, paths, permissions, search, redirections

Louis Ledoux

2026-07-05

Course Organization

Course Organization

Live pacing: about 13 min

Question. What is this course really about, and how will students be evaluated?

Core claim. OPC is not only C syntax: it is the practice of turning text, tools, files, and memory into reliable programs.

Target capability. Students can explain the path from source file to executable and know where Unix, C, TD, and TP fit.

Main Path

- ▶ Start from the question: What is this course really about, and how will students be evaluated?
- ▶ Build the model: OPC is not only C syntax: it is the practice of turning text, tools, files, and memory into reliable programs.
- ▶ End with a checkable action: Students can explain the path from source file to executable and know where Unix, C, TD, and TP fit.
- ▶ Keep only this slide on the horizontal path during the live lecture.



Worked Example

- ▶ Use this as the live trace: Walk from `foo.c` to `./foo`, then ask what each tool has to know before the program can run.
- ▶ Representative source slide: 7
- ▶ Ask students to predict the next state before revealing the explanation.
- ▶ Then connect the result back to the core claim.



Anecdote Or Motivation

Use the first lecture to reset expectations: compilation errors and shell errors are feedback from tools, not random obstacles.

- ▶ Use this only if students need a reason to care.
- ▶ If the room is already following, skip down to the check question.

Check Question

- ▶ Ask for a prediction, not a definition.
- ▶ Require a short justification: command trace, memory drawing, type rule, or file state.
- ▶ If more than a third of the room hesitates, go down to the source backup.

Outline

- Part 1: Linux and Shell scripting
- 2 CM, 1 TD, 2 TP
- Part 2: C Language

- Kahoot time ! (2 Q)
- Goto: <https://kahoot.it/>
- Insert PIN number
- Insert a nickname

#19

Backup Index

Original slides 1-19

UE Outils et programmation en C L3 Informatique

Backup: UE Outils et programmation en C L3 Informatique

Original slide 1

UE Outils et programmation en C L3 Informatique

Backup: \$whoami

Original slide 2

- ▶ Name: Angeliki
- ▶ Surname: Kritikakou
- ▶ Email: angeliki.kritikakou@univ-rennes1.fr
- ▶ Position:
- ▶ Associate Professor (HDR) - Researcher
- ▶ Innovation Chair IUF
- ▶ Teaching: ISTIC mainly L3
- ▶ Research: TARAN team, INRIA-IRISA
- ▶ Research Interests:
- ▶ Computer Architecture, Embedded Systems, Mixed-critical Systems, Fault Tolerance, Low energy design, Task Scheduling and Allocation, Memory Management, Design Space Exploration

\$whoami

- Name: Angeliki
- Surname: Kritikakou
- Email: angeliki.kritikakou@univ-rennes1.fr
- Position:
- Associate Professor (HDR) - Researcher
- Innovation Chair IUF
- Teaching: ISTIC mainly L3
- Research: TARAN team, INRIA-IRISA
- Research Interests:
- Computer Architecture, Embedded Systems, Mixed-critical Systems, Fault Tolerance, Low energy design, Task Scheduling and Allocation, Memory Management, Design Space Exploration



42

Backup: Organisation 2025-2026

Original slide 3

- ▶ CM: Angeliki Kritikakou
- ▶ 11 main courses
- ▶ Interactive course: Kahoot Tests
- ▶ TD: A. Mostefaoui, S. Errabii, F. Santos, C. Viho
- ▶ 3 exercise courses
- ▶ TP: A. Mostefaoui, S. Errabii, F. Santos, C. Viho
- ▶ 13 lab exercises
- ▶ Note: 1/2 Theoretical exam (15% CC-Unix, 35 % CC-C) + 1/2 Final Lab exam (50%)
- ▶ Theoretical exam: Multiple choice questions
- ▶ Final lab exam:
- ▶ Independent
- ▶ 3 November 2025 (Last Lab)
- ▶ Duration: 1h (slot 15:00 or 16:05)

Organisation 2025-2026

CM: Angeliki Kritikakou
11 main courses
Interactive course: Kahoot Tests
TD: A. Mostefaoui, S. Errabii, F. Santos, C. Viho
3 exercise courses
TP: A. Mostefaoui, S. Errabii, F. Santos, C. Viho
13 lab exercises
Note: 1/2 Theoretical exam (15% CC-Unix, 35 % CC-C) + 1/2 Final Lab exam (50%)
Theoretical exam: Multiple choice questions
Final lab exam:
Independent
3 November 2025 (Last Lab)
Duration: 1h (slot 15:00 or 16:05)

43

Backup: Introduction

Original slide 4

- Introduction

Backup: Creation...From a text file to an executable

Original slide 5

- ▶ ?
- ▶ foo.c

```
$ ./foo
i+j=38
$
```

Creation: From a text file to an executable



~foo

```
#include <stdio.h>
int
main()
{
    int i = 12;
    int j = 26;
    int k = i+j;
    printf("i+j=%d\n", k);
    return 0;
}
```

1,1 Top



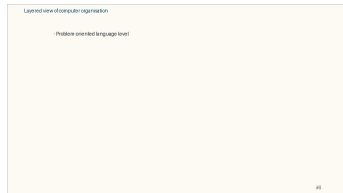
```
$ ./foo
i+j=38
$
```

45

Backup: Layered view of computer organisation

Original slide 6

- ▶ Problem-oriented language level
- ▶ L5



Backup: L5: Problem-oriented language

Original slide 7

- File foo.c is written with the syntax of C language

L5: Problem oriented language

- File foo.c written with the syntax of C language

```
#include <stdio.h>
int main()
{
    int i = 12;
    int j = 21;
    int k = i+j;
    printf("i+j=%d\n", k);
    return 0;
}
```

1,1 Top

47

Backup: Layered view of computer organisation

Original slide 8

- ▶ Problem-oriented language level
- ▶ L5
- ▶ Translation
- ▶ (Compilation)
- ▶ Assembly language level
- ▶ L4



Backup: Layered view of computer organisation

Original slide 10

- ▶ Problem-oriented language level
- ▶ L5
- ▶ Translation
- ▶ (Compilation)
- ▶ Assembly language level
- ▶ L4
- ▶ Translation
- ▶ (Assembler)
- ▶ Operating System Machine level
- ▶ L3



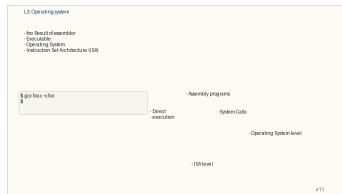
Backup: L3: Operating system

Original slide 11

- ▶ foo: Result of assembler
- ▶ Executable:
- ▶ Operating System
- ▶ Instruction Set Architecture (ISA)
- ▶ Assembly programs

```
$ gcc foo.c -o foo
$
```

- ▶ Direct
- ▶ execution
- ▶ System Calls
- ▶ Operating System level
- ▶ ISA level



Backup: Layered view of computer organisation

Original slide 12

- ▶ Problem-oriented language level
- ▶ L5
- ▶ Translation
- ▶ (Compilation)
- ▶ Assembly language level
- ▶ L4
- ▶ Translation
- ▶ (Assembler)
- ▶ Operating System Machine level
- ▶ L3
- ▶ Partial interpretation
- ▶ (O.S.)
- ▶ Instruction Set Architecture level
- ▶ L2



Backup: L2: Instruction Set Architecture

Original slide 13

- ▶ foo:
- ▶ stored in the instruction memory
- ▶ consists of several machine code instructions
- ▶ depend on the ISA of the used processor (X86, ARM, SPARC, NIOS2...)
- ▶ Processor
- ▶ Data
- ▶ Instruction memory
- ▶ Adresses
- ▶ Results
- ▶ Instructions



Backup: Layered view of computer organisation

Original slide 14

- ▶ Problem-oriented language level
- ▶ L5
- ▶ Translation
- ▶ (Compilation)
- ▶ Assembly language level
- ▶ L4
- ▶ Translation
- ▶ (Assembler)
- ▶ Operating System Machine level
- ▶ L3
- ▶ Partial interpretation
- ▶ (O.S.)
- ▶ Instruction Set Architecture level
- ▶ L2
- ▶ Interpretation
- ▶ (micro-program)



Backup: L1: Micro architecture

Original slide 15

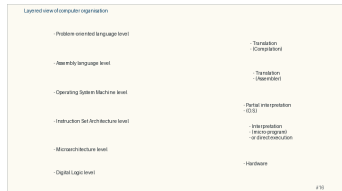
- ▶ Each ISA instruction is implemented by a set of micro-operations (micro-ops)
- ▶ Micro-ops:
 - ▶ Created by the Control Unit
 - ▶ Describe how the Processing Unit has to behave to execute the ISA instruction
- ▶ Data
- ▶ Control Unit
 - ▶ (Finite State Machine)
- ▶ Processing Unit
 - ▶ (operands, registres, etc.)
- ▶ Commands
- ▶ Instruction memory
- ▶ Adresses
- ▶ State
- ▶ Results



Backup: Layered view of computer organisation

Original slide 16

- ▶ Problem-oriented language level
- ▶ L5
- ▶ Translation
- ▶ (Compilation)
- ▶ Assembly language level
- ▶ L4
- ▶ Translation
- ▶ (Assembler)
- ▶ Operating System Machine level
- ▶ L3
- ▶ Partial interpretation
- ▶ (O.S.)
- ▶ Instruction Set Architecture level
- ▶ L2
- ▶ Interpretation
- ▶ (micro-program)



Backup: L0: Digital Logic

Original slide 17

- ▶ Hardware is responsible for everything!
- ▶ Digital gates
- ▶ Boolean values: 0 and 1
- ▶ Data
- ▶ Clk
- ▶ Clk
- ▶ 1
- ▶ 1
- ▶ SRAM
- ▶ 256x8
- ▶ MUX
- ▶ MUX
- ▶ MUX
- ▶ UAL
- ▶ 0
- ▶ 0



Backup: Layered view of computer organisation

Original slide 18

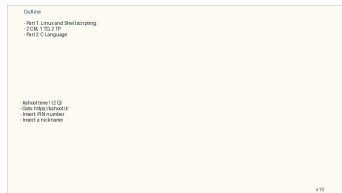
- ▶ Java
- ▶ Python
- ▶ ...
- ▶ C (S5)
- ▶ Problem-oriented language level
- ▶ L5
- ▶ Translation
- ▶ (Compilation)
- ▶ ILM (S6)
- ▶ Assembly language level
- ▶ L4
- ▶ Translation
- ▶ (Assembler)
- ▶ ...
- ▶ Unix (S5)
- ▶ SYRIF (S6)



Backup: Outline

Original slide 19

- ▶ Part 1: Linux and Shell scripting:
 - ▶ 2 CM, 1 TD, 2 TP
- ▶ Part 2: C Language
 - ▶ 9 CM, 2 TD, 10 TP
- ▶ Kahoot time ! (2 Q)
- ▶ Goto: <https://kahoot.it/>
- ▶ Insert: PIN number
- ▶ Insert: a nickname



Linux and OS

Linux and OS

Live pacing: about 8 min

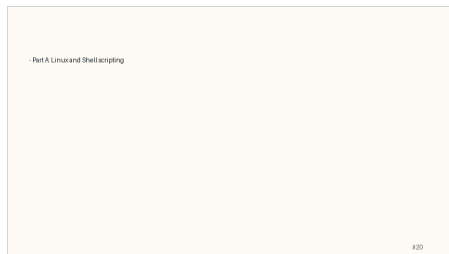
Question. What does the operating system do between the program and the machine?

Core claim. The OS gives a program controlled access to files, processes, memory, and devices.

Target capability. Students can place shell commands, processes, files, and permissions in the OS layer.

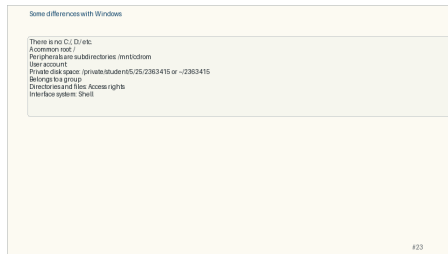
Main Path

- ▶ Start from the question: What does the operating system do between the program and the machine?
- ▶ Build the model: The OS gives a program controlled access to files, processes, memory, and devices.
- ▶ End with a checkable action: Students can place shell commands, processes, files, and permissions in the OS layer.
- ▶ Keep only this slide on the horizontal path during the live lecture.



Worked Example

- ▶ Use this as the live trace: Compare `ls`, an editor, and a compiled program: all are processes asking the OS for services.
- ▶ Representative source slide: 23
- ▶ Ask students to predict the next state before revealing the explanation.
- ▶ Then connect the result back to the core claim.



Anecdote Or Motivation

A useful mental image: user programs are guests; the OS is the host that owns the keys.

- ▶ Use this only if students need a reason to care.
- ▶ If the room is already following, skip down to the check question.

Check Question

- ▶ Ask for a prediction, not a definition.
- ▶ Require a short justification: command trace, memory drawing, type rule, or file state.
- ▶ If more than a third of the room hesitates, go down to the source backup.

Shell

Command Line Interface (CLI)
Input: The commands you want to execute (use the keyboard)
Output: Shell passes the commands to the OS which executes them, and provides the result (usually to the screen)
Scripts: Combination of commands
Variables
Interaction with the Shell: Terminal emulators
Konsole, Xterm, Gnome-terminal...
Several shells are available
sh (Bourne shell), bash (Bourne shell), csh, zsh, ksh...

#25

Backup Index

Original slides 20-25

- Part A: Linux and Shell scripting

Backup: Part A: Linux and Shell scripting

Original slide 20

- Part A: Linux and Shell scripting

Backup: Operating system

Original slide 21

- ▶ Manages:
- ▶ Process: creation, execution, termination, ...
- ▶ Memory: isolation, paging, virtual memory, ...
- ▶ Files: creation, access, permissions, ...
- ▶ Peripherals: usage, common interface, ...
- ▶ Provides a higher layer abstraction to interact with hardware peripherals
- ▶ System calls
- ▶ Focus: Utilisation of the Operating System
- ▶ Linux

Operating system

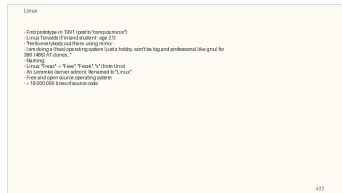
- Manages
 - Process creation, execution, termination, ...
 - Memory isolation, paging, virtual memory, ...
 - File creation, access, permissions, ...
 - Peripherals usage, common interface, ...
- Provides a higher layer abstraction to interact with hardware peripherals
- System calls
- Focus: Utilisation of the Operating System
- Linux

421

Backup: Linux

Original slide 22

- ▶ First prototype in 1991 (post to "comp.os.minix"):
- ▶ Linus Torvalds (Finland student - age 21):
- ▶ "Hello everybody out there using minix -
- ▶ I am doing a (free) operating system (just a hobby, won't be big and professional like gnu) for 386 (486) AT clones..."
- ▶ Naming:
- ▶ Linus: "Freax" -> "Free", "Freak", "x" (from Unix)
- ▶ Ari Lemmke (server admin): Renamed to "Linux"
- ▶ Free and open source operating system
- ▶ > 18.000.000 lines of source code



Backup: Some differences with Windows

Original slide 23

There is no: C:/, D:/ etc.

A common root: /

Peripherals are subdirectories: /mnt/cdrom

User account:

Private disk space: /private/student/5/25/2363415

Belongs to a group

Directories and files: Access rights

Interface system: Shell

Some differences with Windows

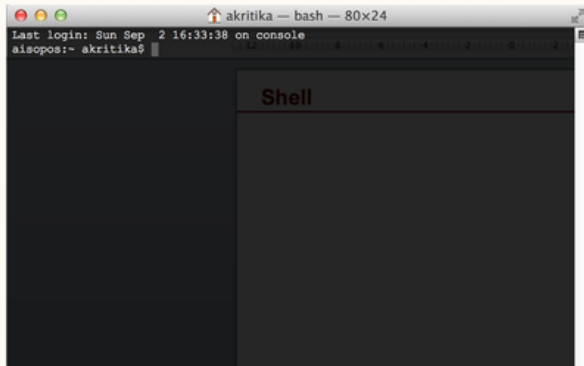
- There is no C:/ D:/ etc.
- A common root: /
- Peripherals are subdirectories: /mnt/cdrom
- User account:
- Private disk space: /private/student/5/25/2363415 or ~/2363415
- Belongs to a group
- Directories and files: Access rights
- Interface system: Shell

423

Backup: Interface with the system: The Shell

Original slide 24

Interface with the system: The Shell



Backup: Shell

Original slide 25

Command Line Interface (CLI)

Input: The commands you want to execute (use the keyboard)

Output: Shell passes the commands to the OS, which executes them, and provides the result (usually to the screen)

Scripts: Combination of commands

Variables

Interaction with the Shell: Terminal emulators

Konsole, Xterm, Gnome-terminal...

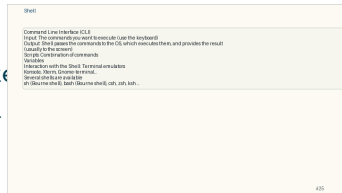
Several shells are available

`sh` (Bourne shell), `bash` (Bourne shell), `csh`, `zsh`, `ksh` ...

Each user has a preselected shell

Selection in the file: `/etc/passwd`

Command to change the shell: `chsh`



Directories and Paths

Directories and Paths

Live pacing: about 8 min

Question. How do commands find data and compose with other commands?

Core claim. Paths, permissions, streams, and redirections are the vocabulary of reproducible command-line work.

Target capability. Students can predict where a command reads, writes, searches, and why it may fail.

Main Path

- ▶ Start from the question: How do commands find data and compose with other commands?
- ▶ Build the model: Paths, permissions, streams, and redirections are the vocabulary of reproducible command-line work.
- ▶ End with a checkable action: Students can predict where a command reads, writes, searches, and why it may fail.
- ▶ Keep only this slide on the horizontal path during the live lecture.

Directories

- Home Directory (\$HOME)
- Each user has a home directory connected with his name
- Name: 2363415
- Home directory: /private/student/5/26/2363415 or ~(/2363415)
- Working Directory (\$PWD)
- Current directory
- When connected Working directory = Home directory
- Current directory:
- Parent directory:
- Root directory /

#26

Worked Example

- ▶ Use this as the live trace: Start with `pwd`, `ls`, `find`, then redirect output into a file and inspect the result.
- ▶ Representative source slide: 27
- ▶ Ask students to predict the next state before revealing the explanation.
- ▶ Then connect the result back to the core claim.

```
Paths
-----
Absolute
From root directory: /private/student5/25/2363415
Relative
From working directory:
Assume the following working directory:
/private/student5/25/2363415
You can access a file in another directory /private/student5/25/2363414 by:
./2363414
```

Anecdote Or Motivation

Most beginner Unix bugs are not algorithmic; they are wrong directory, wrong name, wrong rights, or wrong stream.

- ▶ Use this only if students need a reason to care.
- ▶ If the room is already following, skip down to the check question.

Check Question

- ▶ Ask for a prediction, not a definition.
- ▶ Require a short justification: command trace, memory drawing, type rule, or file state.
- ▶ If more than a third of the room hesitates, go down to the source backup.

Useful information

tab key: One tab key automatically fills the word you are started typing, if exists
Two tab keys: Prints all the existing words in that start with the letters that you have typed
clear command: Clears the screen in the shell
Up arrow key: Shows the previous command
history command: Show all previous commands
!! Executes the previous command
Do not use special characters in the filenames that include also the space character and the hyphen character (-)
Put as default language ENGLISH, the French translation is not complete...

Backup Index

Original slides 26-30

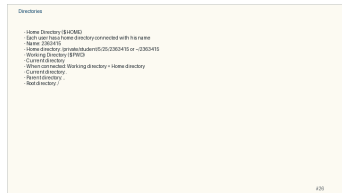
Directories

- Home Directory (\$HOME)
- Each user has a home directory connected with his name
- Name: 2363415
- Home directory: /private/student/5/25/2363415 or ~/2363415
- Working Directory (\$PWD)
- Current directory
- When connected: Working directory = Home directory
- Current directory: .
- Parent directory: ..
- Root directory: /

Backup: Directories

Original slide 26

- ▶ Home Directory (\$HOME)
- ▶ Each user has a home directory connected with his name
- ▶ Name: 2363415
- ▶ Home directory: /private/student/5/25/2363415 or ~/2363415
- ▶ Working Directory (\$PWD)
- ▶ Current directory
- ▶ When connected: Working directory = Home directory
- ▶ Current directory: .
- ▶ Parent directory: ..
- ▶ Root directory: /
- ▶ User home directory: ~



Backup: Paths

Original slide 27

Absolute

From root directory / : /private/student/5/25/2363415

Relative

From working directory:

Assume the following working directory:

/private/student/5/25/2363415

You can access a file in another directory /private/student/5/25/2363414 by:
../2363414

Paths	
Absolute	From root directory /private/student/5/25/2363415
Relative	From working directory
Assume the following working directory:	private/student/5/25/2363415
You can access a file in another directory /private/student/5/25/2363414 by:	../2363414

Backup: Wildcard characters

Original slide 28

Special characters which can be used in combination

One and only one character: **?**

Ex: `ls /etc/rc????`

From 0 to many characters: *****

Ex: `ls /etc/rc*`

Specific characters : **[...]**

Ex: `ls [abc]oo.c`

Wildcard characters

Special characters which can be used in combination with commands

- One and only one character: **?**
Ex: `ls /etc/rc???`
- From 0 to many characters: *****
Ex: `ls /etc/rc*`
- Specific characters: **[...]**
Ex: `ls [abc]oo.c`

430

Backup: How to treat them as normal (by the Shell)

Original slide 29

Preserve the literal value of next character: \
Preserve the literal value of a character sequence
As before, except for the characters \$ and `: "..."
Command replacement: Execution of the command and
`command` (back ticks)
\$(command)



Backup: Useful information

Original slide 30

tab key:

One tab key: automatically fills the word you are s

Two tab keys: Prints all the existing words in that

`clear` command: Cleans the screen in the shell

Up arrow key: Shows the previous command

`history` command: Show all previous commands

`!!`: Executes the previous command

Do not use special characters in the filenames, that include also the space

Put as default language ENGLISH, the French translation is not complete ...



Commands and Files

Commands and Files

Live pacing: about 8 min

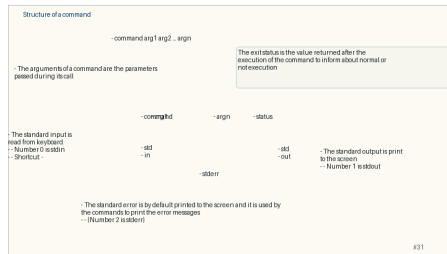
Question. How do commands find data and compose with other commands?

Core claim. Paths, permissions, streams, and redirections are the vocabulary of reproducible command-line work.

Target capability. Students can predict where a command reads, writes, searches, and why it may fail.

Main Path

- ▶ Start from the question: How do commands find data and compose with other commands?
- ▶ Build the model: Paths, permissions, streams, and redirections are the vocabulary of reproducible command-line work.
- ▶ End with a checkable action: Students can predict where a command reads, writes, searches, and why it may fail.
- ▶ Keep only this slide on the horizontal path during the live lecture.



Worked Example

- ▶ Use this as the live trace: Start with `pwd`, `ls`, `find`, then redirect output into a file and inspect the result.
- ▶ Representative source slide: 38
- ▶ Ask students to predict the next state before revealing the explanation.
- ▶ Then connect the result back to the core claim.



Anecdote Or Motivation

Most beginner Unix bugs are not algorithmic; they are wrong directory, wrong name, wrong rights, or wrong stream.

- ▶ Use this only if students need a reason to care.
- ▶ If the room is already following, skip down to the check question.

Check Question

- ▶ Ask for a prediction, not a definition.
- ▶ Require a short justification: command trace, memory drawing, type rule, or file state.
- ▶ If more than a third of the room hesitates, go down to the source backup.



Backup Index

Original slides 31-38

Structure of a command

- command arg1 arg2 ... argn

- The arguments of a command are the parameters passed during its call

The exit status is the value returned after the execution of the command to inform about normal or not execution

- command

- argn

- status

- The standard input is read from keyboard
-- Number 0 is stdin
- Shortcut -

- std
- in

- std
- out

- The standard output is print to the screen
-- Number 1 is stdout

- stderr

- The standard error is by default printed to the screen and it is used by the commands to print the error messages
-- (Number 2 is stderr)

Backup: Structure of a command

Original slide 31

► command arg1 arg2 ... argn

The exit status is the value returned after the exe

► The arguments of a command are the parameters passed during its call

► command

► arg1

► argn

► status

► The standard input is read from keyboard

► ► Number 0 is stdin

► ► Shortcut: -

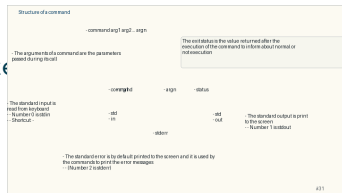
► std

► in

► std

► out

► The standard error is printed to the



Backup: Shell Commands

Original slide 32

- ▶ Directory management
- ▶ File management
- ▶ Redirections
- ▶ Process management
- ▶ Shell programming: Scripts
- ▶ This course does NOT give an exhaustive list of commands and a complete command description ->

Use:

- ▶ ▶ documentation
- ▶ ▶ man or info command
- ▶ ▶ apropos command

Shell Commands

- Directory management
- File management
- Redirections
- Process management
- Shell programming: Scripts
- This course does NOT give an exhaustive list of commands and a complete command description -> Use:
 - documentation
 - man or info command
 - apropos command

432

Backup: Directory management

Original slide 33

Print the full pathname of the working directory: `pwd`

Change directories: `cd`

`cd` or `cd ~`: Go to home directory

`cd path`: Go to directory indicated by path

Creation/Delete directories

`mkdir` dirname: Creates a directory named dirname

`rmdir` dirname: Deletes the directory named dirname (empty)

`rm -r` dirname: Deletes the directory named dirname recursively (subdirectories)

NO TRASHBIN !

Rename: `mv` dirsouce dirdest

Hierarchical copy (subdirectories): `cp -r` dirsouce dirdest



Backup: Listing

Original slide 34

```
ls: Lists the files in the working directory
ls dirname: Lists the content of directory dirname
Some parameters passed as arguments:
-l: Use a long listing format (details)
-a: Not ignore entries starting with . (hidden folders)
```



Backup: File management

Original slide 35

Create file by:

Update access & modification time: `touch filename`

Delete file: `rm filename`

Rename: `mv filesource filedest`

Copy:

`cp filesource filedest`

`cp filesource dirdest`

Remove non-directory suffix part from a filename: `dirname`

Remove the directory part suffix from a filename: `basename`

Count the number of words of a file: `wc -w filename`

File management

Create file by
Update access & modification time: touch filename
Delete file: rm filename
Rename: mv filesource filedest
Copy
cp filesource filedest
cp filesource dirdest
Remove non-directory suffix part from a filename: dirname
Remove the directory part suffix from a filename: basename
Count the number of words of a file: wc -w filename

435

Backup: File management

Original slide 36

Print the contents of a file:

`more filename`: Prints page by page the content of `filename`

`less filename`: Similar to `more`, but it allows to go back

`head -n 42 filename`: Print the first 42 lines of `filename`

`tail -n 42 filename`: Print the last 42 lines of `filename`

Concatenate files (create and view files):

`cat filename1 ... filenameN` (print context of files at stdout)

`cat`: Print at stdout whatever is tapped at stdin (cntrl+D indicates EOF)

`- :` Uses what is tapped with the keyboard instead of an existing file

Difference between two files:

`diff filename1 filename2`



Backup: File management: Links

Original slide 37

- ▶ Associate multiple filenames with a single file
- ▶ Hard: In filename1 filename2
- ▶ New filename (filename2) to access existing file (filename1)
- ▶ Cannot link directories
- ▶ Cannot across file system boundaries
- ▶ Source of link is removed: Link always refers to the source
- ▶ Soft or Symbolic: In -s filename1 filename2
- ▶ A symbolic path indicating the abstract location of another file
- ▶ Can link directories
- ▶ Can across file system boundaries
- ▶ Source of link is removed: Link not updated

File management: Links

- Associate multiple filenames with a single file
- Hard: In filename1 filename2
- New filename (filename2) to access existing file (filename1)
- Cannot link directories
- Cannot across file system boundaries
- Source of link is removed: Link always refers to the source
- Soft or Symbolic: In -s filename1 filename2
- Symbolic path indicating the abstract location of another file
- Can link directories
- Can across file system boundaries

437

Backup: Directory/File management: Examples

Original slide 38

- ▶ Kahoot time !
- ▶ (4 Q)



Permissions and Search

Permissions and Search

Live pacing: about 8 min

Question. How do commands find data and compose with other commands?

Core claim. Paths, permissions, streams, and redirections are the vocabulary of reproducible command-line work.

Target capability. Students can predict where a command reads, writes, searches, and why it may fail.

Main Path

- ▶ Start from the question: How do commands find data and compose with other commands?
- ▶ Build the model: Paths, permissions, streams, and redirections are the vocabulary of reproducible command-line work.
- ▶ End with a checkable action: Students can predict where a command reads, writes, searches, and why it may fail.
- ▶ Keep only this slide on the horizontal path during the live lecture.

Directory/File permissions

-rwx	-rwx	-rwx
"---	"---	"---
-owned by user	-group	-others

Change permissions: `chmod`
`chmod [who] [op permissions] + name`

- Who	- Op	- Permissions
- Owned by user (u)	- Add (+)	- Read or List (r)
- Group (g)	- Remove (-)	- Write (w)
- Others (o)	- Define (=)	- Execute or Access (x)
- Nothing (all)		

#39

Worked Example

- ▶ Use this as the live trace: Start with `pwd`, `ls`, `find`, then redirect output into a file and inspect the result.
- ▶ Representative source slide: 42
- ▶ Ask students to predict the next state before revealing the explanation.
- ▶ Then connect the result back to the core claim.

```
File management Search files

Search files based on criteria (print path):
find directory [criteria]
The search is performed to directory (recursive to subdirectories)
Apply a command at the found files option -exec cmd {} \;
Apply the command cmd in each found file {} (it replaces the @)
End of the command \;

Some search criteria
-name filename: based on the filename (-iname: ignore case)
-user username: based on the user that owns the file
-size number[k|M] based on size [k: KB, M: MB]
-type [t][d] based on the type [directory, file, link]
-mtime x based on time of modification (less than x days)

Criteria can be combined: -o -a -not
```

Anecdote Or Motivation

Most beginner Unix bugs are not algorithmic; they are wrong directory, wrong name, wrong rights, or wrong stream.

- ▶ Use this only if students need a reason to care.
- ▶ If the room is already following, skip down to the check question.

Check Question

- ▶ Ask for a prediction, not a definition.
- ▶ Require a short justification: command trace, memory drawing, type rule, or file state.
- ▶ If more than a third of the room hesitates, go down to the source backup.

File management Search in files

Print all lines of the files of the directory that contain a word. grep string directory

The string can be expressed as a regular expression

Whatever character: .

Repetition of pattern: *

Interval of characters: [a-z]

Return value: 0 found, else 1=0

Examples

```
grep -e test ./
grep "m+?" filename
grep "tot(1-3)" filename
```

#45

Backup Index

Original slides 39-45

Directory/File permissions

- rwx

- rwx

- rwx

- ---

- ---

- ---

- owned by user

- group

- others

Change permissions: chmod
chmod [who] op permissions [op permissions] * name

- Who:
- Owned by user (u)
- Group (g)
- Others (o)
- Nothing (all)

- Op:
- Add (+)
- Remove (-)
- Define (=)

- Permissions:
- Read or List (r)
- Write (w)
- Execute or Access(x)

Backup: Directory/File permissions

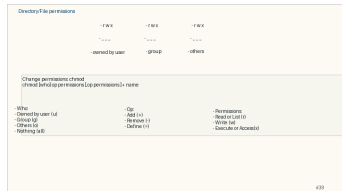
Original slide 39

- ▶ r w x
- ▶ r w x
- ▶ r w x
- ▶ _ _ _
- ▶ _ _ _
- ▶ _ _ _
- ▶ group
- ▶ others
- ▶ owned by user

Change permissions: chmod

`chmod [who] op permissions [,op permissions ]* name`

- ▶ Who:
- ▶ Owned by user (u)
- ▶ Group (g)
- ▶ Others (o)



Backup: Directory/File permissions

Original slide 40

- ▶ r w x
- ▶ r w x
- ▶ r w x
- ▶ _ _ _
- ▶ _ _ _
- ▶ _ _ _
- ▶ group
- ▶ others
- ▶ owned by user
- ▶ Permissions can be given with numeric code per field
- ▶ In a field, you put 1 or 0 in each column to give or not the corresponding permission.
- ▶ 1 1 1
- ▶ 1 1 0
- ▶ 1 0 0
- ▶ group

Directory/File permissions

-rwx	-rwx	-rwx
user	group	others
owned by user	group	others
-111	-110	-100
owned by user	group	others
-7	-0	-4

chmod 704 filename

#40

Backup: Directory/File permissions: Examples

Original slide 41

- ▶ Kahoot time !
- ▶ (3 Q)



Backup: File management: Search files

Original slide 42

Search files based on criteria (print path):

`find directory [criteria]`

The search is performed to directory (recursive to

Apply a command at the found files: option `-exec` c

Apply the command `cmd` in each found file (it replac

End of the command: `\;`

Some search criteria:

`-name filename`: based on the filename (`-iname`: ignore case)

`-user username`: based on the user that owns the file

`-size number[c|k|M]`: based on size [B, KB, MB]

`-type [d|f|l]`: based on the type [directory, file, link]

`-mtime x`: based on time of modification (less than x days)

....

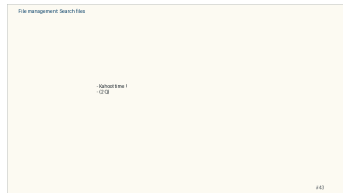
Criteria can be combined: `-o -a -not`



Backup: File management: Search files

Original slide 43

- ▶ Kahoot time !
- ▶ (2 Q)



Backup: File management: Search files: Examples

Original slide 44

```
find /usr -name "*.c"
find /usr -user angeliki
find . -not \( -name "*.obj" -o -name "*.map" \)
find . \( -user angeliki -a -name "*.c" \)
find . -name "*.java" -exec lpr {} \;
```

File management Search files Examples

```
find /usr -name "*.c"
find /usr -user angeliki
find -not \( -name "*.obj" -o -name "*.map" \)
find -i /usr -user angeliki -a -name "*.c"
find -name "*.java" -exec lpr {} \;
```

444

Backup: File management: Search in files

Original slide 45

Print all lines of the files of the directory that

The string can be expressed as a regular expression

Whatever character : .

Repetition of pattern: *

Interval of characters: [x-y]

Return value: 0 found, else !=0

Examples:

```
grep -r test ./
```

```
grep "m*t" filename
```

```
grep "toto[1-3]" filename
```

File management: Search in files

Print all lines of the files of the directory that contain a word (grepating directory)

The string can be expressed as a regular expression

Whatever character : .

Repetition of pattern: *

Interval of characters [x-y]

Return value: 0 found, else !=0

Examples:

```
grep -r test ./
grep "m*t" filename
grep "toto[1-3]" filename
```

#45

Redirections

Redirections

Live pacing: about 8 min

Question. How do commands find data and compose with other commands?

Core claim. Paths, permissions, streams, and redirections are the vocabulary of reproducible command-line work.

Target capability. Students can predict where a command reads, writes, searches, and why it may fail.

Main Path

- ▶ Start from the question: How do commands find data and compose with other commands?
- ▶ Build the model: Paths, permissions, streams, and redirections are the vocabulary of reproducible command-line work.
- ▶ End with a checkable action: Students can predict where a command reads, writes, searches, and why it may fail.
- ▶ Keep only this slide on the horizontal path during the live lecture.

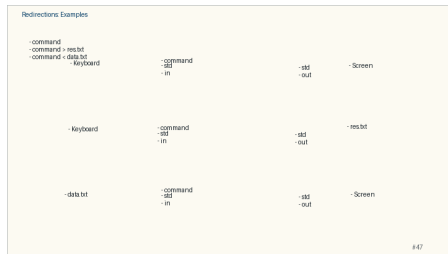
Redirections

- Redirection of standard input (stdin):
 - Use the content of a file as input to a command (instead of keyboard)
 - `command < filename`
- Redirection of standard output (stdout):
 - Print the output of a command into a file (instead of screen)
 - `command > filename` (overwrite old content of file)
 - `command >> filename` (append to the content of file)
- Pipe: redirect standard output to standard input of commands
 - Use the output of a command as input to another command
 - A subshell is created to execute `command_2`
 - `command_1 | command_2`

#45

Worked Example

- ▶ Use this as the live trace: Start with `pwd`, `ls`, `find`, then redirect output into a file and inspect the result.
- ▶ Representative source slide: 47
- ▶ Ask students to predict the next state before revealing the explanation.
- ▶ Then connect the result back to the core claim.



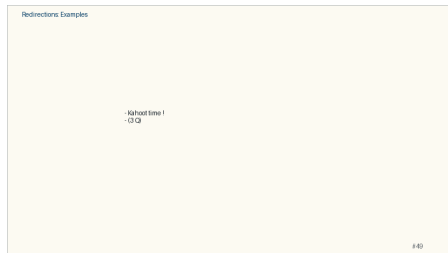
Anecdote Or Motivation

Most beginner Unix bugs are not algorithmic; they are wrong directory, wrong name, wrong rights, or wrong stream.

- ▶ Use this only if students need a reason to care.
- ▶ If the room is already following, skip down to the check question.

Check Question

- ▶ Ask for a prediction, not a definition.
- ▶ Require a short justification: command trace, memory drawing, type rule, or file state.
- ▶ If more than a third of the room hesitates, go down to the source backup.



Backup Index

Original slides 46-49

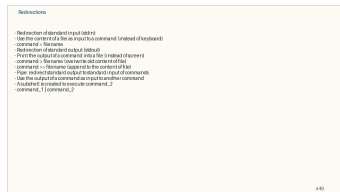
Redirections

- Redirection of standard input (stdin):
- Use the content of a file as input to a command (instead of keyboard)
- `command < filename`
- Redirection of standard output (stdout)
- Print the output of a command into a file (instead of screen)
- `command > filename` (overwrite old content of file)
- `command >> filename` (append to the content of file)
- Pipe: redirect standard output to standard input of commands
- Use the output of a command as input to another command
- A subshell is created to execute `command_2`
- `command_1 | command_2`

Backup: Redirections

Original slide 46

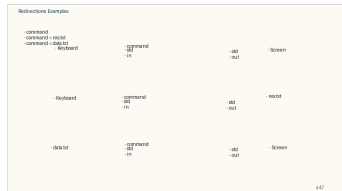
- ▶ Redirection of standard input (stdin):
- ▶ Use the content of a file as input to a command (instead of keyboard)
- ▶ `command < filename`
- ▶ Redirection of standard output (stdout)
- ▶ Print the output of a command into a file (instead of screen)
- ▶ `command > filename` (overwrite old content of file)
- ▶ `command » filename` (append to the content of file)
- ▶ Pipe: redirect standard output to standard input of commands
- ▶ Use the output of a command as input to another command
- ▶ A subshell is created to execute `command_2`
- ▶ `command_1 | command_2`



Backup: Redirections: Examples

Original slide 47

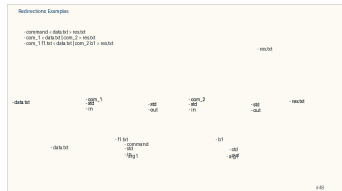
- ▶ `command`
- ▶ `command > res.txt`
- ▶ `command < data.txt`
- ▶ `command`
- ▶ `Keyboard`
- ▶ `std`
- ▶ `in`
- ▶ `Screen`
- ▶ `std`
- ▶ `out`
- ▶ `res.txt`
- ▶ `command`
- ▶ `Keyboard`
- ▶ `std`
- ▶ `in`
- ▶ `std`



Backup: Redirections: Examples

Original slide 48

- ▶ `command < data.txt > res.txt`
- ▶ `com_1 < data.txt | com_2 > res.txt`
- ▶ `com_1 f1.txt < data.txt | com_2 b1 > res.txt`
- ▶ `res.txt`
- ▶ `com_1`
- ▶ `com_1`
- ▶ `com_2`
- ▶ `com_2`
- ▶ `res.txt`
- ▶ `res.txt`
- ▶ `data.txt`
- ▶ `data.txt`
- ▶ `std`
- ▶ `in`
- ▶ `std`
- ▶ `in`



Backup: Redirections: Examples

Original slide 49

- ▶ Kahoot time !
- ▶ (3 Q)

